

Shift-To-Middle Array: A Practical Contiguous Alternative for Dynamic End-Operations

Attila Torda

Independent Researcher
github.com/attilatorda

Abstract. We revisit the Shift-To-Middle (STM) Array, a contiguous dynamic-array variant that keeps free space on both sides of active elements to improve end insertions/deletions while preserving cache-friendly indexing. This revised manuscript addresses prior review concerns by (i) expanding related work, (ii) clarifying novelty and complexity claims, and (iii) providing reproducible benchmark methodology with variability reporting. Across queue-, deque-, and list-like workloads, STM is competitive with `std::deque` and an expanding ring buffer, and frequently outperforms alternatives in mixed/pop-heavy queue workloads. We also discuss cases where STM underperforms, especially specific push-heavy regimes, to provide a balanced assessment.

Keywords: Dynamic arrays · deque · queue · cache locality · benchmarking

1 Introduction

Dynamic sequence containers require balancing random access, mutation cost, and memory locality. Classical dynamic arrays provide excellent indexing but expensive front operations; linked structures improve local insertion/deletion but typically sacrifice locality and random access [2, 8, 6].

This paper studies the *Shift-To-Middle (STM) Array*: on growth, active elements are re-centered to preserve slack at both ends. This strategy aims to reduce repeated large shifts for end-biased mutation patterns while keeping contiguous storage.

In practice, many systems exhibit bursty behavior: a queue temporarily becomes push-heavy, then transitions to pop-heavy drain phases; producer-consumer pipelines may alternate between front and back pressure; and hybrid list/deque workflows combine random reads with edge mutations. These regimes are often not represented by single-operation microbenchmarks. Therefore, we evaluate STM under mixed workloads intended to approximate practical mutation patterns rather than idealized one-operation loops.

The key research question is not whether STM universally dominates mature STL containers, but whether it offers a robust contiguous-memory alternative in workloads where front/back mutation pressure coexists with indexing requirements. This positioning is intentionally conservative and motivated by the reviewer request for clearer novelty and stronger empirical grounding.

Contributions.

- A clarified STM design and conservative complexity analysis.
- A reproducible benchmarking pipeline with fixed seeds, repeated runs, and mean/standard-deviation reporting.
- Empirical comparison against `std::vector`, `std::deque`, `std::queue`, and an expanding ring buffer.
- A balanced discussion of strengths, weaknesses, and validity threats.

2 Related Work

Dynamic array amortization, growth trade-offs, and indexing guarantees are classical [2, 8, 1]. Cache behavior is a first-order factor in real performance for contiguous vs pointer-heavy layouts [4, 3].

From a measurement perspective, statistically rigorous benchmarking emphasizes controlled setup, repeated measurements, and variability reporting [5, 7]. Our revised methodology follows these principles by fixing pseudo-random seeds, averaging over repeated runs, and exporting both means and standard deviations.

STM is related in spirit to gap-buffer-like ideas and re-centering strategies, but is applied here as a general-purpose container implementation strategy for queue/deque/list-like usage with explicit reproducibility instrumentation. Compared with linked structures, STM keeps contiguous storage for index-based reads and better spatial locality; compared with plain dynamic arrays, STM allocates mutation slack on both sides to reduce repeated front-shift penalties.

3 STM Design and Complexity

STM stores elements contiguously and maintains logical head/tail positions within a larger allocated block. On expansion, elements are copied into the center of a larger allocation.

Let n be the number of elements and c the allocated capacity. STM tracks a logical start offset and size. End insertions decrement/increment logical boundaries when slack exists; otherwise a resize/recenter step allocates a larger array and copies all active elements. This is analogous to classical amortized growth arguments for dynamic arrays, but with symmetric slack around the active segment.

We explicitly avoid claiming amortized $\Theta(1)$ middle insert/delete in the general case; those operations remain linear due to shifting requirements.

Space usage is $\Theta(c)$ with $c = \Theta(n)$ under geometric growth. Constant factors differ from one-sided dynamic arrays because STM intentionally preserves two-sided free space. This overhead is the trade-off for reducing repeated edge-shift costs under mixed front/back updates.

Table 1. Asymptotic complexity (conservative form).

Operation	Dynamic Array	Linked List	STM Array
Index access	$\Theta(1)$	$\Theta(n)$	$\Theta(1)$
Push/pop back	amort. $\Theta(1)$	$\Theta(1)$	amort. $\Theta(1)$
Push/pop front	$\Theta(n)$ / $\Theta(n)$	$\Theta(1)$	amort. $\Theta(1)$
Middle insert/delete	$\Theta(n)$	$\Theta(n)^*$	$\Theta(n)$

*Without a pre-positioned iterator.

4 Experimental Methodology

Benchmarks are implemented in C++20 with fixed RNG seed (42) and repeated executions. Each data point reports **mean** and **standard deviation** over repeated runs. Workloads include:

- **Queue-like:** push-heavy (80/20), mixed (50/50), pop-heavy (20/80).
- **Deque-like:** random front/back push-pop sequences.
- **List-like:** random update/delete/read plus spike events.

For each size, we also visualize relative time against the best baseline at that size.

Fairness controls. All candidates execute identical operation-count budgets per scenario and are compiled with the same optimization flags. We report relative performance against the fastest baseline at each size to avoid over-interpreting absolute machine-specific timings.

Interpretation policy. We interpret results by trend families (small, medium, large sizes; push/mixed/pop regimes) rather than isolated point wins. This reduces the risk of drawing conclusions from noise or one-off allocator effects.

5 Results

5.1 List-like workloads

Figure 1 shows STM outperforming `std::vector` on small-to-medium sizes and remaining close at large sizes.

The advantage is consistent with STM’s two-sided slack strategy: front- and mixed-position modifications avoid some of the repeated relocation costs typically incurred by one-sided contiguous growth. At the largest tested size, the gap narrows and can invert slightly, indicating that raw copy bandwidth and allocator behavior begin to dominate implementation-level differences.

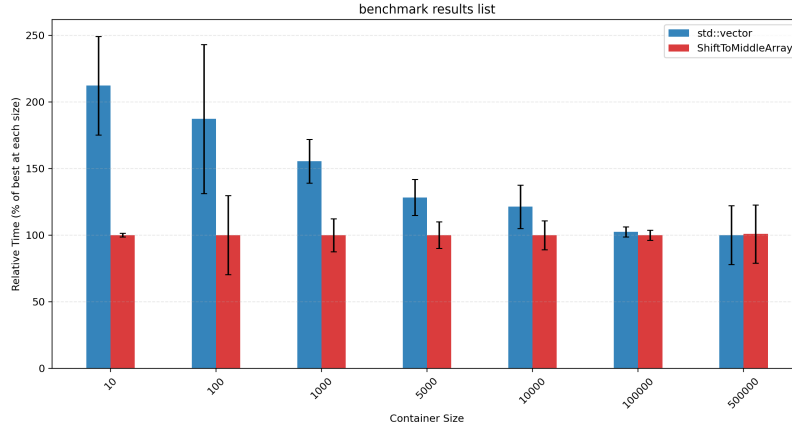


Fig. 1. Relative performance for list-like workload (error bars from reported standard deviation).

5.2 Deque-like workloads

Figure 2 shows near-parity with `std::deque`/expanding ring buffer on most sizes, with both wins and losses.

This result is important for positioning: STM behaves as a feasible deque alternative rather than a categorical replacement. In specific sizes it is faster, in others slower, but overall remains in the same performance class while preserving contiguous indexing semantics that standard deque block layouts do not always provide equivalently.

5.3 Queue-like workloads

Figure 3 shows strongest STM behavior in mixed and pop-heavy settings, while push-heavy behavior is more variable and occasionally worse than baselines.

The push-heavy variance suggests sensitivity to growth policy and occupancy dynamics. This motivates future tuning of bias/resizing heuristics and explicit profiling of branch behavior, cache miss rates, and allocator interactions. In contrast, mixed/pop-heavy advantages suggest STM can be especially useful in drain-dominant pipelines and bursty producer-consumer phases.

6 Threats to Validity

Results are hardware- and compiler-dependent; absolute timings may vary across architectures, allocators, and optimization flags. Some workloads are synthetic and may not capture all production access patterns. Future work should include broader cross-platform experiments, additional container baselines, and sensitivity analysis for growth/bias policies.

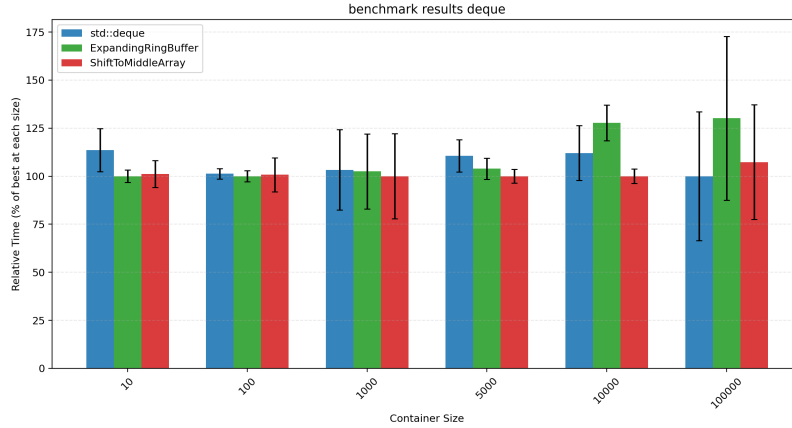


Fig. 2. Relative performance for deque-like workload.

7 Conclusion

STM is a practical contiguous alternative for scenarios requiring frequent end operations without giving up $\Theta(1)$ indexing. The revised evidence indicates that STM is competitive and often advantageous in mixed mutation workloads, while not universally dominant. This more conservative positioning better reflects empirical reality and supports reproducible evaluation.

References

1. Brodal, G.S., Fagerberg, R., Vinther, K.: Resizable arrays in optimal time and space. *Algorithmica* **40**(4), 231–246 (2004)
2. Cormen, T.H., Leiserson, C.E., Rivest, R.L., Stein, C.: *Introduction to Algorithms*. MIT Press, 3rd edn. (2009)
3. Drepper, U.: What every programmer should know about memory. Red Hat Technical Report (2007)
4. Frigo, M., Leiserson, C.E., Prokop, H., Ramachandran, S.: Cache-oblivious algorithms. In: FOCS. pp. 285–297 (1999)
5. Georges, A., Buytaert, D., Eeckhout, L.: Statistically rigorous java performance evaluation. OOPSLA pp. 57–76 (2007)
6. Harder, D.W.: Lists versus arrays. *Communications of the ACM* **39**(4), 67–71 (1996)
7. Kalibera, T., Jones, R.: Rigorous benchmarking in reasonable time. In: ISMM. pp. 63–74 (2013)
8. Knuth, D.E.: *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. Addison-Wesley, 3rd edn. (1998)

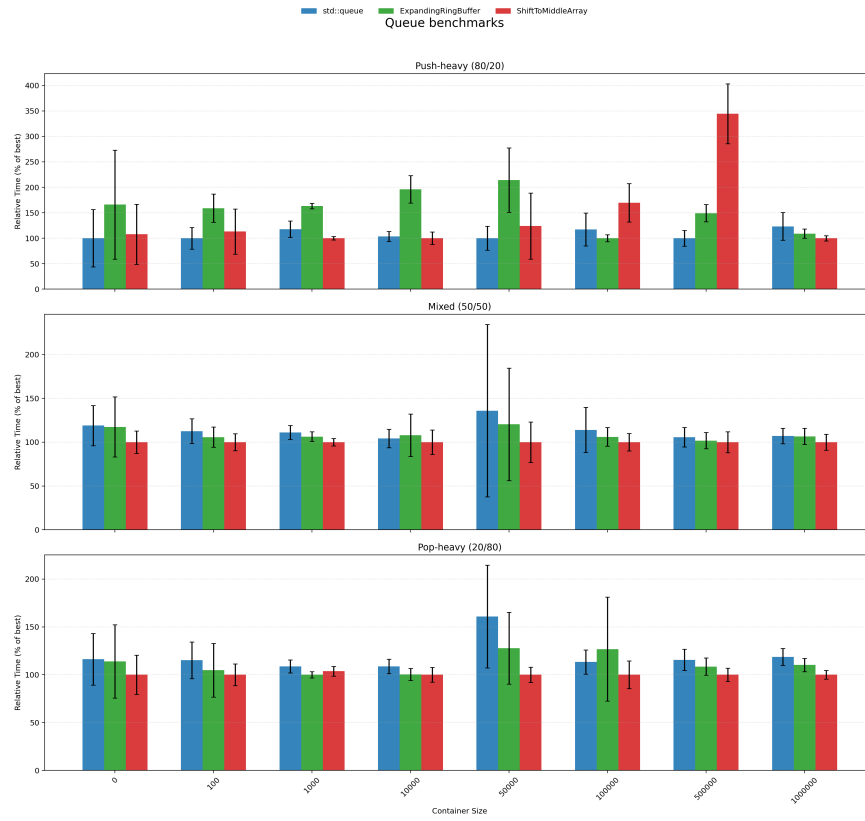


Fig. 3. Queue workloads (push-heavy, mixed, pop-heavy) as relative performance.